

Java Programming: Core Concepts & Operations

Comprehensive Teaching Guide & Practice Assessment Module

TOPICS COVERED: TYPE CASTING, ESCAPE SEQUENCES, CHARACTER MATH, STRINGS & I/O

1. Type Casting & Numeric Conversions

Type casting in Java is the process of converting a variable from one data type to another. Java supports two main types of conversions: **implicit (widening) casting** and **explicit (narrowing) casting**.

1.1 Primitive Numeric Conversions & Rules

When assigning a value of a larger range data type (such as `int`) to a variable of a smaller range type (such as `byte` or `short`), explicit casting is mandatory. Failing to do so results in a compilation error because potential precision loss may occur.

```
int i = 1;
byte b = i; // COMPILER ERROR: Explicit casting required!
byte b2 = (byte)i; // CORRECT: Explicit cast
```

Special Literal Exception: As long as an *integer literal* falls within the permissible range of the target variable (e.g., -128 to 127 for `byte`), explicit casting is **not required** when assigning a literal directly to a `byte` or `short` variable.

1.2 Practical Application: Formatting Decimals

Explicit casting is frequently used to truncate floating-point values or format numbers to specific precision. Below is an example demonstrating sales tax calculation formatted to two decimal places using explicit integer casting:

```
import java.util.Scanner;

public class SalesTax {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter purchase amount: ");
        double purchaseAmount = input.nextDouble();

        double tax = purchaseAmount * 0.06;

        // Truncate decimal digits beyond 2 places using cast to int
        System.out.println("Sales tax is $" + (int)(tax * 100) / 100.0);
    }
}
```

Topic 1 Assessment: Type Casting

Q1. Which of the following statements regarding implicit casting in Java is true? EASY

- A) Converting `double` to `int` happens automatically.
- B) Assigning an `int` literal like `50` to a `byte` variable requires explicit cast `(byte)50`.
- C) Java automatically widens smaller types (like `int` to `double`) without explicit cast syntax.
- D) Explicit casting is never required in Java.

Answer: C | Explanation: Widening conversions (e.g., `int` to `double`) occur automatically via implicit casting.

Q2. What is the output of `System.out.println((int)(12.85 * 100) / 100.0);` ? MEDIUM

- A) 12.85
- B) 12.8
- C) 12.0
- D) 1285.0

Answer: A | Explanation: $12.85 * 100 = 1285.0$. Cast to `int` yields 1285. $1285 / 100.0$ yields double 12.85.

Q3. Given `double d = 1000.34; int i = (int)d;`, what is the value of `i` ? EASY

- A) 1000.34
- B) 1000
- C) 1001
- D) Compilation error

Answer: B | Explanation: Casting double to `int` truncates the fractional decimal portion completely.

Q4. What happens when compiling: `int i = 5; byte b = i;` ? MEDIUM

- A) Works fine because 5 fits in byte.
- B) Compilation error: possible loss of precision.
- C) Runtime exception.
- D) `b` is assigned 0.

Answer: B | Explanation: Assigning an `int` variable (even if value is small) to a `byte` variable requires explicit cast `(byte)i`.

Q5. What is the value of `(int)(float) 1000.34f` in Java? HARD

- A) 1000
- B) 1000.34
- C) Compile-time error
- D) 1000.0

Answer: A | Explanation: Float `1000.34f` is explicitly cast to `int`, dropping fractional part resulting in integer 1000.

2. Escape Characters & Unicode Representation

In Java, certain characters cannot be typed directly into string literals (such as double quotes or line breaks). Java provides **escape sequences**—special notations starting with a backslash (`\`) followed by a character or sequence.

2.1 Common Escape Characters Table

An escape character is interpreted by the compiler as a single unified character entity.

Escape Sequence	Character Name	Unicode Representation	Decimal Value
<code>\b</code>	Backspace	<code>\u0008</code>	8
<code>\t</code>	Tab	<code>\u0009</code>	9
<code>\n</code>	Linefeed (New Line)	<code>\u000A</code>	10
<code>\f</code>	Formfeed	<code>\u000C</code>	12
<code>\r</code>	Carriage Return	<code>\u000D</code>	13
<code>\"</code>	Backslash	<code>\\</code>	92
<code>\"</code>	Double Quote	<code>\"</code>	34

2.2 Usage Example

To print text containing quotation marks, use `\"`:

```
// Error: System.out.println("He said "Java is fun");
System.out.println("He said "Java is fun");
// Output: He said "Java is fun"
```

Topic 2 Assessment: Escape Characters

Q1. Which escape character represents a tab space in Java?

EASY

- A) ☐
- B) ☐
- C) ☐
- D) ☒ \

Answer: B | Explanation: `\t` is the escape sequence for horizontal tab.

Q2. What statement correctly outputs: `\ "` to the console?

MEDIUM

- A) `System.out.println("\ ");`
- B) `System.out.println("\ "');`
- C) `System.out.println("\"");`
- D) `System.out.println(" ");`

Answer: A | Explanation: `\\` escapes the backslash and `\"` escapes the double quote.

Q3. What is the Unicode escape representation for a double quote (`"`)?

MEDIUM

- A) ☐
- B) ☒ \
- C) ☐ "
- D) ☐

Answer: C | Explanation: Unicode `\"` corresponds to decimal 34 (Double Quote).

Q4. What is the output of `System.out.println("A B C");` ?

EASY

- A) A B C on one line
- B) A, followed by new line with B and tab-separated C
- C) Compilation error
- D) A B C printed literally

Answer: B | Explanation: `\n` inserts a newline after A, and `\t` adds a tab between B and C.

Q5. How many characters does Java compiler consider `"\"` to be?

HARD

- A) 2 characters
- B) 1 character
- C) 0 characters
- D) 4 characters

Answer: B | Explanation: `\"` is an escaped backslash sequence representing a single character value (decimal 92).

3. Character Operations & Arithmetic

In Java, characters (`char`) are backed by 16-bit Unicode numerical values. Because of this, binary numeric operators can be performed directly on character variables.

3.1 Character Conversions & Arithmetic

When a character is involved in arithmetic operations, its Unicode numerical value is used. For instance, Unicode for character `'a'` is `97`, and `'A'` is `65`.

```
byte b = 'a'; // Implicit cast allowed (97 fits into byte)
int i = 'a'; // i becomes 97

char c = 90; // c becomes 'Z' (Unicode 90)
char c2 = (char)97; // c2 becomes 'a'
```

3.2 Increment and Decrement on Chars

Increment (`++`) and decrement (`--`) operators work directly on `char` variables without requiring explicit casting.

```
public class CharArithTest {
    public static void main(String[] args) {
        char x = 'a'; // 97
        char y = 'c'; // 99

        System.out.println(++x); // Output: 'b' (x updated to 'b' / 98)
        System.out.println(y++); // Output: 'c' (prints 'c', then y becomes 'd' / 100)
        System.out.println(x - y); // Output: -2 (98 - 100 = -2)
    }
}
```

Topic 3 Assessment: Character Arithmetic

Q1. Given `char c = 'A';` (Unicode 65), what is `(int)c` ? **EASY**

- A) 'A'
- B) 65
- C) 97
- D) Error

Answer: B | Explanation: Casting 'A' to int returns its underlying Unicode value 65.

Q2. Evaluate: `int j = '1' + '2';` (Note: Unicode for '1' is 49, '2' is 50). **MEDIUM**

- A) 3
- B) "12"
- C) 99
- D) 12

Answer: C | Explanation: '1' (49) + '2' (50) evaluates to integer addition: 49 + 50 = 99.

Q3. What is the value of `'b' / 'a'` **in integer arithmetic?** **MEDIUM**

- A) 1
- B) 0
- C) 1.01
- D) Compile error

Answer: A | Explanation: 'b' (98) / 'a' (97) using integer division equals 1.

Q4. If `char c = 'a';`, what does `c++` evaluate to when printed, and what is `c` after? **MEDIUM**

- A) Prints 'b', c becomes 'b'
- B) Prints 'a', c becomes 'b'
- C) Prints 97, c becomes 98
- D) Error

Answer: B | Explanation: Post-increment `c++` returns original value 'a' first, then increments c to 'b'.

Q5. What is the value of expression: `'4' - '3'` ? **HARD**

- A) '1'
- B) 1
- C) 7
- D) 52

Answer: B | Explanation: Unicode for '4' is 52, '3' is 51. 52 - 51 = 1 (int value 1).

4. The String Class & String Concatenation

The `String` class in Java represents a sequence of characters. Unlike `char`, `String` is a **reference type** (a predefined class in `java.lang` package).

4.1 String Concatenation Rules

The `+` operator serves as string concatenation whenever at least one operand is a `String`:

- If **one operand is a String**, non-string operands are automatically converted to strings and concatenated.

- If **neither operand is a String**, `+` acts as standard addition.
- Concatenation evaluates left-to-right unless overridden by parentheses `()`.

```
String message = "Welcome " + "to " + "Java"; // "Welcome to Java"
String s = "Chapter" + 2;                      // "Chapter2"
String s1 = "Supplement" + 'B';                 // "SupplementB"

int i = 1, j = 2;
System.out.println("i + j is " + i + j);        // Output: "i + j is 12"
System.out.println("i + j is " + (i + j));        // Output: "i + j is 3"
```

Operator Precedence Notice: In `"1" + 1 + 1`, left-to-right evaluation yields `"11" + 1 = "111"`. But in `1 + 1 + "1"`, addition occurs first: `2 + "1" = "21"`.

Topic 4 Assessment: String Concatenation

Q1. What is the output of `System.out.println("1" + 1 + 1);` ? **EASY**

- A) 3
- B) 111
- C) 12
- D) Compilation error

Answer: B | Explanation: Left-to-right: "1" + 1 -> "11", then "11" + 1 -> "111".

Q2. What is the output of `System.out.println("1" + (1 + 1));` ? **MEDIUM**

- A) 111
- B) 12
- C) 3
- D) 13

Answer: B | Explanation: Parentheses evaluated first: (1 + 1) = 2. Then "1" + 2 = "12".

Q3. What is the result of `1 + "Welcome " + 1 + 1` ? **MEDIUM**

- A) "1Welcome 2"
- B) "1Welcome 11"
- C) "3Welcome "
- D) Error

Answer: B | Explanation: 1 + "Welcome " = "1Welcome ". Then "1Welcome " + 1 = "1Welcome 1", then + 1 = "1Welcome 11".

Q4. What is the result of `1 + "Welcome " + (1 + 1)` ? **EASY**

- A) "1Welcome 11"
- B) "1Welcome 2"
- C) "2Welcome 2"
- D) "3Welcome "

Answer: B | Explanation: (1+1)=2. 1 + "Welcome " = "1Welcome ". "1Welcome " + 2 = "1Welcome 2".

Q5. Evaluate: `1 + "Welcome " + 'a' + 1` (where 'a' is char). **HARD**

- A) "1Welcome a1"
- B) "1Welcome 971"
- C) "98Welcome 1"
- D) "1Welcome 98"

Answer: A | Explanation: 1 + "Welcome " = "1Welcome ". "1Welcome " + 'a' = "1Welcome a". "1Welcome a" + 1 = "1Welcome a1".

5. Console Input & Scanner Class Operations

To read string input from the console, Java provides the `Scanner` class (in `java.util`).

5.1 `next()` vs `nextLine()` Methods

- `next()` : Reads a token bounded by **whitespace characters** (space ' ', tab , newline , formfeed , carriage return). It stops reading at the first whitespace.
- `nextLine()` : Reads an entire line of text until the user presses the Enter key.


```
Scanner input = new Scanner(System.in);

// Reading words (delimited by spaces)
String s1 = input.next();
String s2 = input.next();

// Reading an entire line
String fullLine = input.nextLine();
```

Important Caution: To avoid unexpected input skipping, do not call `nextLine()` immediately after calls like `nextInt()`, `nextDouble()`, or `next()` without consuming the leftover newline character in the buffer.

Topic 5 Assessment: Scanner Input

Q1. Which Scanner method reads an entire line of text up to the Enter key? EASY

- A) `next()`
- B) `nextLine()`
- C) `readString()`
- D) `nextWord()`

Answer: B | Explanation: `nextLine()` reads up to the end of the line including spaces.

Q2. If user enters "Welcome to Java" , what does `input.next()` store in variable? MEDIUM

- A) "Welcome to Java"
- B) "Welcome"
- C) "Welcome to"
- D) "Java"

Answer: B | Explanation: `next()` reads delimited by whitespace, so it reads only the first word "Welcome".

Q3. Which of the following is NOT considered a whitespace character by Scanner `next()` ? MEDIUM

- A) Space (' ')
- B) Linefeed (`\n`)
- C) Semicolon (`;`)
- D) Tab (`\t`)

Answer: C | Explanation: Semicolon is standard text, not a whitespace delimiter.

Q4. What common issue occurs when invoking `nextLine()` after `nextInt()` ? HARD

- A) Compiler error
- B) `nextLine()` consumes the leftover newline character immediately and returns an empty string
- C) Scanner throws `NullPointerException`
- D) Program hangs permanently

Answer: B | Explanation: `nextInt()` leaves newline in buffer, which `nextLine()` reads immediately as empty input.

Q5. To read 3 space-separated words into `s1`, `s2`, `s3` , which sequence is required? EASY

- A) Three calls to `input.next()`
- B) One call to `input.nextLine()`
- C) Three calls to `input.nextInt()`
- D) Three calls to `input.nextLine()`

Answer: A | Explanation: Each `next()` call retrieves one whitespace-delimited word.

6. String-to-Number Parsing & Conversion

When input is retrieved from GUI dialog boxes (like `JOptionPane`) or string inputs, numeric values are received as `String` objects (e.g., `"123"`). To perform arithmetic on them, they must be parsed into numeric primitive types.

6.1 Utility Wrapper Classes & Methods

Java provides wrapper classes in the `java.lang` package (automatically imported) containing parsing static methods:

- Convert to `int` : `Integer.parseInt(intString)`
- Convert to `double` : `Double.parseDouble(doubleString)`

```
String intStr = "123";  
int intValue = Integer.parseInt(intStr); // intValue = 123  
  
String doubleStr = "123.45";  
double doubleValue = Double.parseDouble(doubleStr); // doubleValue = 123.45
```

Package Info: Both `Integer` and `Double` belong to `java.lang`, so explicit `import` statements are not required. Passing a non-numeric string (e.g., `"abc"`) will throw a `NumberFormatException` at runtime.

Topic 6 Assessment: String-to-Number Parsing

Q1. Which method converts the string "456" to an integer primitive in Java? EASY

- A) `Integer.parseInt("456")`
- B) `(int)"456"`
- C) `String.toInt("456")`
- D) `Convert.toInt("456")`

Answer: A | Explanation: `Integer.parseInt()` is the standard static method to convert numeric String to int.

Q2. Which package contains the `Integer` and `Double` wrapper classes? EASY

- A) `java.util`
- B) `java.io`
- C) `java.lang`
- D) `java.text`

Answer: C | Explanation: `java.lang` is automatically imported in all Java files.

Q3. What occurs if you execute `Integer.parseInt("12.34");` ? MEDIUM

- A) Returns 12
- B) Throws `NumberFormatException`
- C) Returns 12.34
- D) Compilation error

Answer: B | Explanation: "12.34" contains a decimal point, which cannot be parsed as a pure integer string by `parseInt`.

Q4. What is the correct way to parse "99.9" into a floating-point number? EASY

- A) `Double.parseDouble("99.9")`
- B) `Integer.parseDouble("99.9")`
- C) `(double)"99.9"`
- D) `Scanner.parseDouble("99.9")`

Answer: A | Explanation: `Double.parseDouble()` converts valid floating point strings to double values.

Q5. What is the result of `Integer.parseInt("10") + Double.parseDouble("20.5")` ? HARD

- A) 30.5
- B) "1020.5"
- C) 30
- D) Runtime error

Answer: A | Explanation: `Integer.parseInt("10")` yields int 10, `Double.parseDouble("20.5")` yields double 20.5. $10 + 20.5 = 30.5$.